NAME – SUBASH G
REG NO – 192572150

COURSE NAME – Operation System
COURSE CODE – CSA0408

Lab Observations

EXPERIMENT - 17

Illustrate the deadlock avoidance concept by simulating Banker's algorithm using C.

PROGRAM:

```
#include <stdio.h>

#define MAX_PROCESSES 5
#define MAX_RESOURCES 3

int available[MAX_RESOURCES] = {3, 3, 2};

int maximum[MAX_PROCESSES][MAX_RESOURCES] = {
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2},
    {4, 3, 3}
};

int allocation[MAX_PROCESSES][MAX_RESOURCES] = {
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
    {2, 1, 1},
    {0, 0, 2}
};

int is_safe()
{
    int work[MAX_RESOURCES];
    int finish[MAX_PROCESSES] = {0};
```

```
    int count = 0;

    for (int i = 0; i < MAX_RESOURCES; i++)
        work[i] = available[i];

    while (count < MAX_PROCESSES)
    {
        int found = 0;

        for (int i = 0; i < MAX_PROCESSES; i++)
        {
            if (finish[i] == 0)
            {
                int j;

                for (j = 0; j < MAX_RESOURCES; j++)
                {
                    int need = maximum[i][j] - allocation[i][j];

                    if (need > work[j])
                        break;
                }

                if (j == MAX_RESOURCES)
                {
                    for (int k = 0; k < MAX_RESOURCES; k++)
                        work[k] += allocation[i][k];

                    finish[i] = 1;
                    found = 1;
                    count++;
                }
            }
        }

        if (found == 0)
            return 0;
    }

    return 1;
}

int request_resources(int process_num, int request[])
```

```
{
    for (int i = 0; i < MAX_RESOURCES; i++)
    {
        int need = maximum[process_num][i]
                   - allocation[process_num][i];

        if (request[i] < 0 ||
            request[i] > need ||
            request[i] > available[i])
            return 0;
    }

    for (int i = 0; i < MAX_RESOURCES; i++)
    {
        available[i] -= request[i];
        allocation[process_num][i] += request[i];
    }

    if (is_safe())
        return 1;

    for (int i = 0; i < MAX_RESOURCES; i++)
    {
        available[i] += request[i];
        allocation[process_num][i] -= request[i];
    }

    return 0;
}

int main()
{
    int process_num, request[MAX_RESOURCES];

    printf("Enter process number (0 to 4): ");
    scanf("%d", &process_num);

    if (process_num < 0 || process_num >= MAX_PROCESSES)
    {
        printf("Invalid process number.\n");
        return 1;
    }
```

```
    printf("Enter resource request (e.g., 0 1 0): ");

    for (int i = 0; i < MAX_RESOURCES; i++)
        scanf("%d", &request[i]);

    if (request_resources(process_num, request))
        printf("Request granted. System is in a safe state.\n");
    else
        printf("Request denied. Request is invalid or unsafe.\n");

    return 0;
}
```

OUTPUT:

